Particle Filtering with Lie-Group Camera Projection for Online Multi-Camera UAV Tracking

Siddharth Anand*

This paper studies online tracking of a small unmanned aerial vehicle using multiple calibrated external cameras. The implemented estimator compares a MEKF-style error-state extended Kalman filter, an iterated EKF, their gated variants, and a bootstrap particle filter. All filters use the same Lie-group camera model: a world point is transformed by a calibrated $\mathrm{SE}(3)$ camera extrinsic and projected through a pinhole intrinsic matrix. Monte Carlo simulations in a Gazebo/Purdue-world style camera geometry show that the EKF variants are fastest and most accurate in the nearly unimodal nominal cases, while the particle filter is more appropriate when the visual posterior is multimodal. A candidate-set experiment demonstrates this regime: the particle filter achieves 0.186 m RMSE, compared with 0.319 m for the best gated EKF baseline, because it marginalizes over multiple plausible detections instead of committing to a single association. A hardware-calibration hypothesis test is also performed by replaying an extracted PURT motion in simulation while injecting camera intrinsic and extrinsic errors. The ablation shows that using a mismatched 1600×1200 intrinsic matrix on 640×480 images, or treating an active-marker body frame as an optical camera frame, can reproduce multi-meter particle-filter errors. Finally, a future extension is formulated in which each particle carries a latent $\mathrm{SE}_2(3)$ inertial-navigation state so that body-frame acceleration readings enter the prediction step through the $\mathrm{SO}(3)$ exponential map and group action, while the reported output remains the UAV trajectory.

Nomenclature

$p_{w,k}$	UAV position in the world frame at time index k
$v_{w,k}$	UAV velocity in the world frame at time index k
$z_{j,k}$	Pixel measurement from camera j at time index k
K_j	Intrinsic matrix of camera j
$T_{j,cw}$	World-to-camera extrinsic transform for camera j, an element of $\mathrm{SE}(3)$
$R_{j,cw}, t_{j,cw}$	Rotation and translation blocks of $T_{j,cw}$
$\pi(\cdot)$	Pinhole camera projection map
N_p	Number of particles in the particle filter
$w_k^{(i)}$	Weight of particle i at time index k
$\omega_{m,k}$	Body-frame gyroscope measurement
$a_{m,k}$	Body-frame accelerometer measurement or specific force
$R_{wb,k}$	Body-to-world attitude of the UAV
$\mathcal{V}_k$	Set of cameras with valid detections at time index k

I. Introduction

External vision sensors are attractive for tracking small UAVs in laboratory, urban-canyon, and GPS-denied settings, including Purdue urban-canyon style external-vision work.[6] A multi-camera system can provide multiple simultaneous bearing-like measurements, but its accuracy depends on camera intrinsics, camera-to-world extrinsics, image timing, and detector quality. The estimator must also handle missing views, noisy detections, and ambiguous visual associations.

*M.S. Student, School of Aeronautics and Astronautics, Purdue University, West Lafayette, IN.

This project investigates such a system in the context of online UAV trajectory estimation. The current implemented state is position and velocity,

$$x_k = \begin{bmatrix} p_{w,k} \\ v_{w,k} \end{bmatrix} \in \mathbb{R}^6,$$

while the camera model is written using SE(3) transformations. The implemented filters are a MEKF-style error-state EKF, an iterated EKF (ItEKF), gated versions of both EKF variants, and a bootstrap particle filter (PF). Particle filters are emphasized because they can represent non-Gaussian and multimodal posteriors,[1] which are natural in camera-based tracking when detections are ambiguous.

The paper has four objectives. First, it presents the common Lie-group projection model used by all filters. Second, it compares the EKF-style estimators and PF in controlled Monte Carlo simulations. Third, it identifies a regime in which PF is the more appropriate estimator: ambiguous candidate-set measurements. Fourth, it reproduces a hardware failure hypothesis in simulation by injecting camera calibration errors observed in a PURT hardware data collection. A proposed extension then moves more Lie-group structure into the PF prediction step by carrying latent attitude and using body-frame acceleration readings.

II. Problem Formulation

A. Online Multi-Camera Tracking Task

At each time k, the estimator receives detections from up to N_c calibrated cameras. The objective of the implemented system is to estimate $p_{w,k}$, with $v_{w,k}$ carried to improve prediction between camera updates. The camera detections are two-dimensional pixel measurements; the filters do not receive ground-truth 3D position during online operation.

The controlled simulation uses a 12 s trajectory with time step $\Delta t = 0.05$ s, $N_p = 450$ particles for the baseline PF, and 12 Monte Carlo runs per scenario. The main scenarios are summarized in Table 1. The final stress case includes missing camera detections and false detections, and is kept as a transparency check with both ungated and gated EKF variants.

Table 1. Monte Carlo scenarios used for the filter comparison.

Scenario	Pixel noise [px]	Initial position offset [m]	Dropout	False detections
Nominal	8	$[0.7, -0.5, 0.25]$	0.00	0.00
Poor initialization	8	$[2.5, -2.0, 0.9]$	0.00	0.00
High noise	22	$[0.9, -0.7, 0.25]$	0.00	0.00
Dropout stress	12	$[1.0, -0.8, 0.35]$	0.18	0.06

B. Lie-Group Camera Projection

For camera j, the world-to-camera extrinsic is

$$T_{j,cw} = \begin{bmatrix} R_{j,cw} & t_{j,cw} \\ 0 & 1 \end{bmatrix} \in \mathrm{SE}(3).$$

For a world-frame point $p_{w,k}$, define the homogeneous point

$$\bar{p}_{w,k} = \begin{bmatrix} p_{w,k} \\ 1 \end{bmatrix}.$$

The point is transformed into camera coordinates by the SE(3) action

$$\bar{p}_{j,c,k} = T_{j,cw}\bar{p}_{w,k}, \qquad p_{j,c,k} = \begin{bmatrix} X_{j,c,k} \\ Y_{j,c,k} \\ Z_{j,c,k} \end{bmatrix}.$$

The predicted pixel location is

$$\hat{z}_{j,k} = h_j(x_k) = \pi(K_j, T_{j,cw}\bar{p}_{w,k}),$$

where

$$\pi(K_j, p_{j,c,k}) = \begin{bmatrix} f_{x,j} X_{j,c,k}/Z_{j,c,k} + c_{x,j} \\ f_{y,j} Y_{j,c,k}/Z_{j,c,k} + c_{y,j} \end{bmatrix}.$$

The measurement model is

$$z_{j,k} = h_j(x_k) + n_{j,k}, \qquad n_{j,k} \sim \mathcal{N}(0, R_j).$$

This is the primary Lie-group component of the implemented filters: camera extrinsics are represented as SE(3) elements, and all filters use the same world-to-camera action before applying the nonlinear projection map familiar from multiple-view geometry.[3]

III. Implemented Filters

A. Shared Prediction Model

The implemented Monte Carlo comparison uses a constant-velocity process model:

$$x_{k+1} = F_k x_k + q_k, \qquad F_k = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0 & I_3 \end{bmatrix}.$$

Equivalently,

$$p_{w,k+1} = p_{w,k} + \Delta t\, v_{w,k} + q_{p,k}, \qquad v_{w,k+1} = v_{w,k} + q_{v,k}.$$

Because $x_k \in \mathbb{R}^6$, the prediction state is an additive abelian group. Writing this portion in Lie-group notation does not change the implemented equations. The nontrivial implemented Lie-group component is therefore the SE(3) camera measurement model above.

B. MEKF-Style EKF

The MEKF-style filter propagates a mean $\hat{x}_k$ and covariance P_k. The predicted covariance is

$$P_k^- = F_k P_{k-1} F_k^\mathsf{T} + Q_k.$$

For a valid camera j, the residual is

$$r_{j,k} = z_{j,k} - h_j(\hat{x}_k^-).$$

The projection Jacobian with respect to p_w is

$$J_{\pi,j,k} = \begin{bmatrix} f_{x,j}/Z & 0 & -f_{x,j}X/Z^2 \\ 0 & f_{y,j}/Z & -f_{y,j}Y/Z^2 \end{bmatrix},$$

where $p_{j,c,k} = [X, Y, Z]^\mathsf{T}$. Thus

$$H_{j,k} = \begin{bmatrix} J_{\pi,j,k} R_{j,cw} & 0_{2\times 3} \end{bmatrix}.$$

Stacking all valid cameras gives r_k, H_k, and R_k. The correction is

$$S_k = H_k P_k^- H_k^\mathsf{T} + R_k, \qquad K_k = P_k^- H_k^\mathsf{T} S_k^{-1},$$

$$\hat{x}_k = \hat{x}_k^- + K_k r_k,$$

with the Joseph covariance update used for numerical robustness.

C. Iterated EKF and Gated Variants

The ItEKF uses the same prediction and camera model, but repeatedly relinearizes $h(\cdot)$ around the current update estimate. At iteration ℓ,

$$\hat{z}_k^{(\ell)} = h(\hat{x}_k^{(\ell)}), \qquad H_k^{(\ell)} = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_k^{(\ell)}}.$$

The iterated residual is

$$\tilde{r}_k^{(\ell)} = z_k - h(\hat{x}_k^{(\ell)}) + H_k^{(\ell)}(\hat{x}_k^{(\ell)} - \hat{x}_k^{-}),$$

and the update is

$$\hat{x}_k^{(\ell+1)} = \hat{x}_k^{-} + K_k^{(\ell)}\tilde{r}_k^{(\ell)}.$$

The gated variants reject a camera update when its Mahalanobis distance exceeds a selected threshold:

$$r_{j,k}^{\mathsf{T}} S_{j,k}^{-1} r_{j,k} > \gamma.$$

This is a measurement-validation stage, not a dropout fix: if a camera does not provide a measurement, both gated and ungated variants simply skip that camera.

D. Bootstrap Particle Filter

The PF represents the posterior by weighted samples

$$\mathcal{X}_k = \left\{x_k^{(i)}, w_k^{(i)}\right\}_{i=1}^{N_p}, \qquad x_k^{(i)} = \begin{bmatrix} p_{w,k}^{(i)} \\ v_{w,k}^{(i)} \end{bmatrix}.$$

Particles are predicted through the same constant-velocity model with sampled process noise. For each valid camera,

$$\hat{z}_{j,k}^{(i)} = \pi(K_j, T_{j,cw}\bar{p}_{w,k}^{(i)}), \qquad r_{j,k}^{(i)} = z_{j,k} - \hat{z}_{j,k}^{(i)}.$$

The particle likelihood is

$$\ell_k^{(i)} = \prod_{j \in \mathcal{V}_k} \mathcal{N}(r_{j,k}^{(i)}; 0, R_j),$$

with a robust likelihood floor in contaminated measurement cases. The weights are updated by

$$w_k^{(i)} \propto w_{k|k-1}^{(i)} \ell_k^{(i)}, \qquad \sum_i w_k^{(i)} = 1.$$

Particles are resampled with probability $w_k^{(i)}$, and the reported trajectory estimate is

$$\hat{p}_{w,k} = \sum_{i=1}^{N_p} w_k^{(i)} p_{w,k}^{(i)}.$$

IV. Simulation Results

A. Trajectory and Visual Setup

Figure 1 shows the simulated top-view trajectory and camera layout used for the controlled comparison. The same geometry is used by all filters, and all filters receive the same generated detections.

B. Filter Accuracy and Runtime

Table 2 reports RMSE, 90th percentile error, and mean runtime per step. The MEKF and ItEKF have nearly identical accuracy in nominal and high-noise cases because the posterior remains close to unimodal. The ItEKF is more expensive because it repeatedly relinearizes the projection model. The PF is slower than the EKFs at the baseline 450 particles and is not the most accurate in easy cases, but it provides a sampled posterior that becomes valuable when measurements are ambiguous.

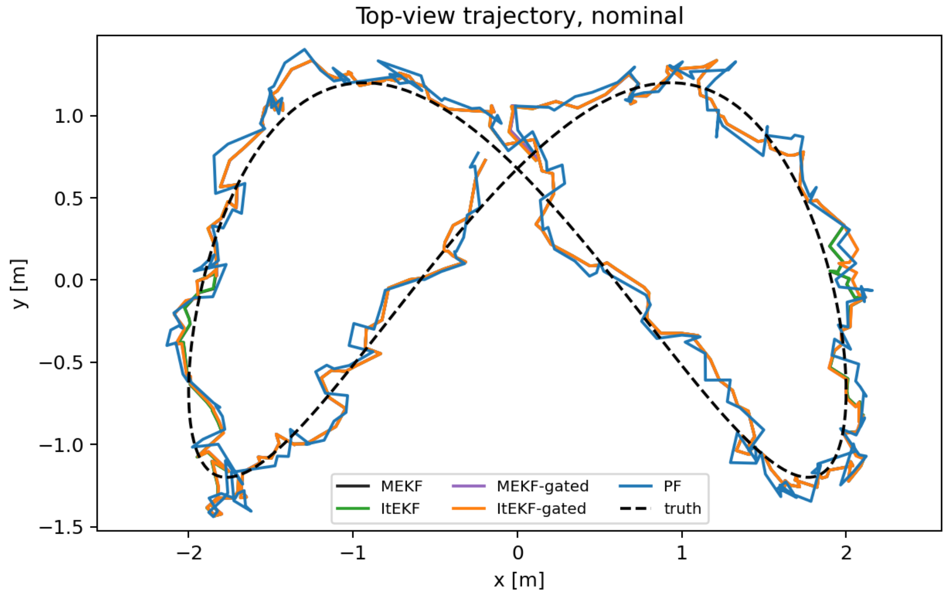


Figure 1. Top-view simulated trajectory and external-camera layout used in the controlled Monte Carlo comparison.

Table 2. Main Monte Carlo comparison. Runtime is mean step time.

Scenario	Filter	RMSE [m]	P90 [m]	Runtime [ms]
Nominal	MEKF	0.153	0.221	0.170
Nominal	ItEKF	0.153	0.221	0.489
Nominal	MEKF-gated	0.154	0.223	0.348
Nominal	ItEKF-gated	0.154	0.222	1.017
Nominal	PF	0.183	0.265	1.039
Poor initialization	MEKF	0.155	0.224	0.179
Poor initialization	ItEKF	0.155	0.224	0.480
Poor initialization	MEKF-gated	0.156	0.226	0.347
Poor initialization	ItEKF-gated	0.155	0.225	1.008
Poor initialization	PF	0.215	0.304	1.026
High noise	MEKF	0.346	0.506	0.207
High noise	ItEKF	0.347	0.508	0.567
High noise	MEKF-gated	0.347	0.511	0.388
High noise	ItEKF-gated	0.349	0.513	1.168
High noise	PF	0.446	0.651	1.142
Dropout stress	MEKF	0.539	0.830	0.193
Dropout stress	ItEKF	0.542	0.828	0.540
Dropout stress	MEKF-gated	0.248	0.361	0.363
Dropout stress	ItEKF-gated	0.249	0.362	1.048
Dropout stress	PF	0.317	0.465	1.075

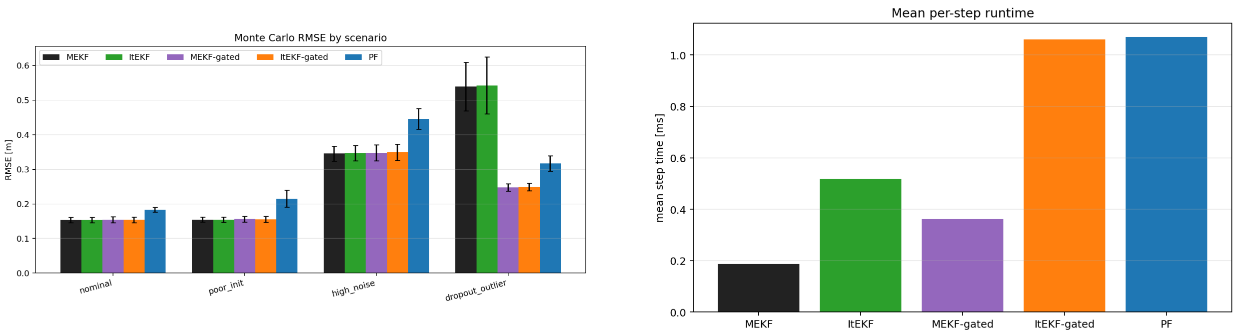


Figure 2. RMSE and computation-time comparison across implemented filter variants.

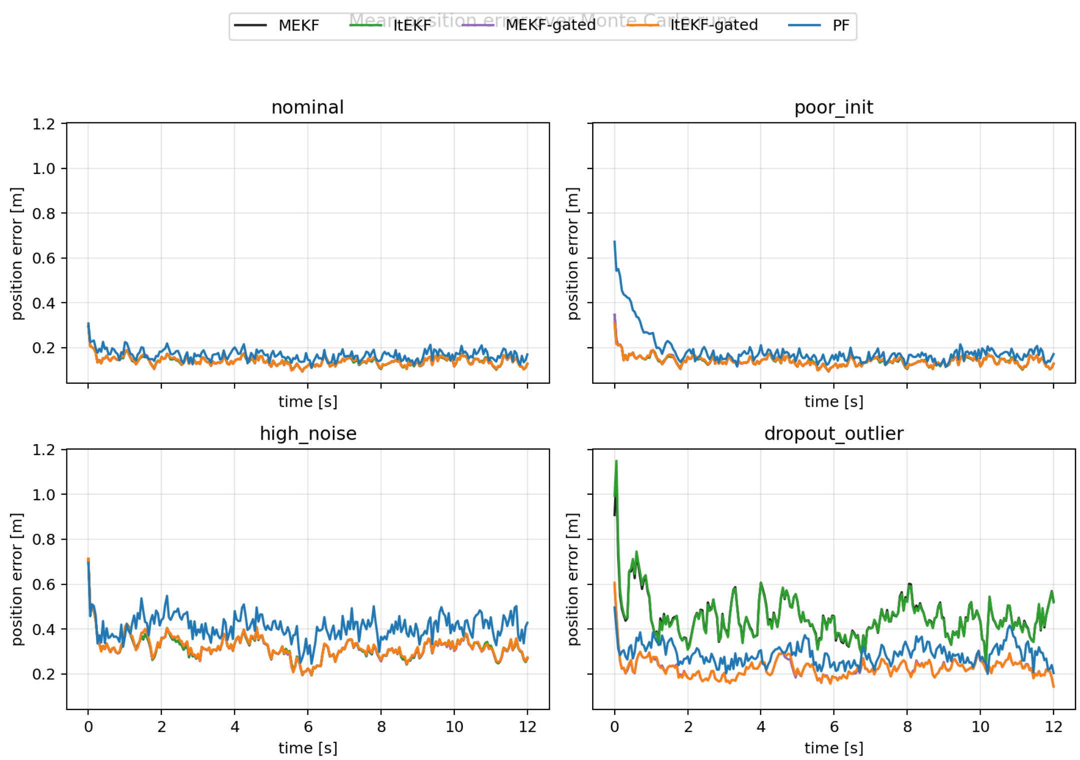


Figure 3. Representative time history of position error for the simulated comparison.

C. Particle Count Sensitivity

The PF improves as the particle count increases, but the improvement saturates in mostly unimodal cases while runtime grows. Table 3 shows the dropout stress sweep. Increasing from 450 to 14400 particles improves RMSE from 0.317 m to 0.287 m, while runtime rises from 0.897 ms to 14.055 ms per step. This supports the interpretation that PF should not be claimed as universally superior by simply increasing particle count; its strength is the ability to represent non-Gaussian uncertainty.

Table 3. PF particle-count sweep in the dropout stress case.

Particles	RMSE [m]	P90 [m]	Runtime [ms]
225	0.319	0.459	0.684
450	0.317	0.465	0.897
900	0.296	0.434	1.319
1800	0.292	0.431	2.065
3600	0.289	0.419	4.167
7200	0.288	0.419	7.747
14400	0.287	0.415	14.055

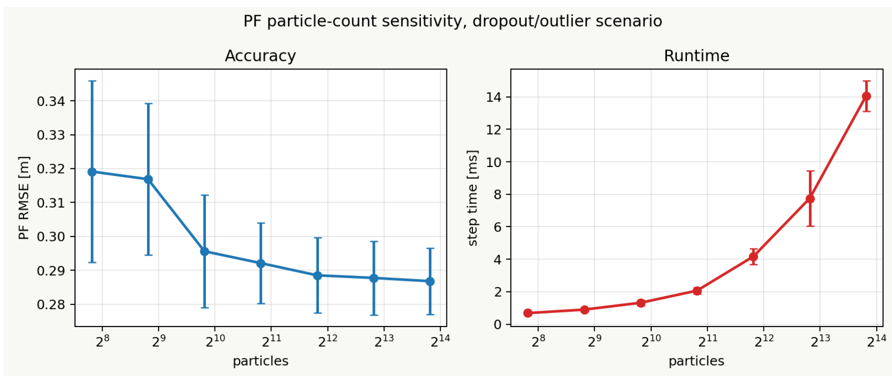


Figure 4. PF particle-count sensitivity. More particles reduce sampling error but increase runtime.

V. A Case Where PF Is Preferable

The most favorable case for PF is not a simple high-noise Gaussian measurement model, but an ambiguous visual association model. Suppose each camera returns a candidate set

$$\mathcal{Z}_{j,k} = \{z_{j,k}^{(1)}, \ldots, z_{j,k}^{(M_j)}\}.$$

An EKF-style filter generally needs a single association

$$\tilde{z}_{j,k} = z_{j,k}^{(m^\star)}$$

before applying a Gaussian update. If $m^\star$ is wrong, the innovation can pull the filter toward the wrong target. The PF can instead use a mixture likelihood:

$$L_{j,k}(x_k^{(i)}) = \epsilon + \sum_{m=1}^{M_j} \alpha_m \mathcal{N}\left(z_{j,k}^{(m)}; h_j(x_k^{(i)}), R_j\right),$$

$$w_k^{(i)} \propto w_{k|k-1}^{(i)} \prod_j L_{j,k}(x_k^{(i)}).$$

No candidate label is assigned to a particle. Instead, each particle is weighted by how well it explains any plausible candidate.

Table 4 shows that this is the regime where PF clearly wins. The candidate-set PF with 450 particles achieves 0.186 m RMSE, while the best gated EKF baseline is 0.319 m. This is the main evidence that PF is the natural estimator when the posterior is multimodal.

Table 4. Ambiguous candidate-set experiment.

Filter	RMSE [m]	P90 [m]	Runtime [ms]
MEKF	0.541	0.784	0.145
ItEKF	0.542	0.784	0.382
MEKF-gated	0.319	0.476	0.257
ItEKF-gated	0.325	0.485	0.705
PF-candidates	**0.186**	**0.266**	0.969

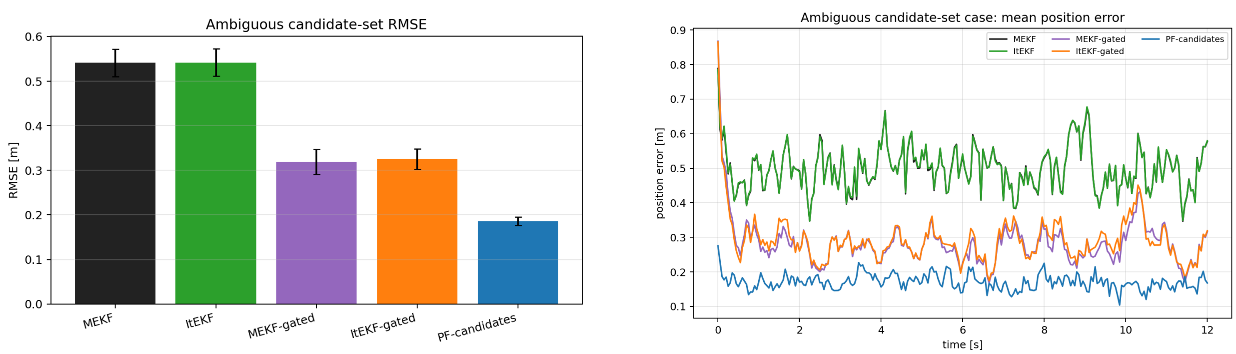


Figure 5. Ambiguous candidate-set case. The PF uses a mixture likelihood over candidates and maintains multiple hypotheses.

VI. Hardware Calibration Hypothesis

A. Motivation

A PURT hardware test on April 24, 2026 produced very large raw PF errors. The working hypothesis was that the active-marker rigid body gave a stable QTM marker pose, but this pose was not the optical camera pose used by the projection model. In addition, an old `camera0.yaml` file described a 1600×1200 calibration, while the UP camera images used by the tracker were 640×480. Either error corrupts the SE(3) projection model.

The hypothesis was tested in simulation by replaying an extracted PURT run-1 QTM trajectory and generating clean camera measurements from a correct camera rig. The PF was then deliberately run with wrong intrinsics and/or wrong extrinsics. This isolates calibration error from detector quality, ROS timing, and QTM parsing.

B. Calibration Ablation

The correct UP intrinsics were approximately

$$K_{\mathrm{UP}} = \begin{bmatrix} 338.972 & 0 & 320.473 \\ 0 & 340.546 & 235.618 \\ 0 & 0 & 1 \end{bmatrix},$$

whereas the erroneous `camera0.yaml` intrinsic matrix was

$$K_{\text{bad}} = \begin{bmatrix} 847.429 & 0 & 801.181 \\ 0 & 851.364 & 589.046 \\ 0 & 0 & 1 \end{bmatrix}.$$

The wrong-extrinsic cases perturb the assumed camera pose as

$$\hat{R}_{j,wc} = R_{j,wc}\,\mathrm{Exp}(\delta\phi_j^{\wedge}), \qquad \hat{t}_{j,wc} = t_{j,wc} + R_{j,wc}\delta c_j,$$

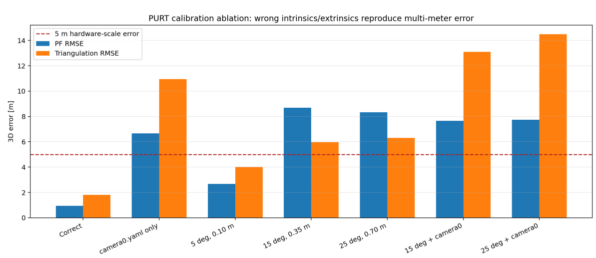
then convert to world-to-camera form

$$\hat{R}_{j,cw} = \hat{R}_{j,wc}^{\mathsf{T}}, \qquad \hat{t}_{j,cw} = -\hat{R}_{j,wc}^{\mathsf{T}}\hat{t}_{j,wc}.$$

Table 5 and Fig. 6 show that the hypothesis is plausible. The clean case gives 0.943 m PF RMSE. Using only `camera0.yaml` raises the PF RMSE to 6.654 m. A 15 deg, 0.35 m marker-like optical-frame error raises it to 8.692 m. Combining the same marker-like error with `camera0.yaml` gives 7.650 m. Thus the calibration failure mode alone is sufficient to reproduce multi-meter hardware-scale error in an otherwise clean simulator.

Table 5. Hardware calibration hypothesis ablation.

Case	Intrinsics	Extrinsic error	PF RMSE [m]	PF P90 [m]	Pixel P90 [px]
Correct	UP	correct	0.943	1.351	17.1
`camera0.yaml` only	bad	correct	6.654	9.055	632.3
5 deg, 0.10 m	UP	mild	2.679	4.010	38.8
15 deg, 0.35 m	UP	marker-like	8.692	15.089	112.1
25 deg, 0.70 m	UP	severe	8.331	13.671	197.6
15 deg + `camera0`	bad	marker-like	7.650	10.662	738.5
25 deg + `camera0`	bad	severe	7.740	11.064	847.0



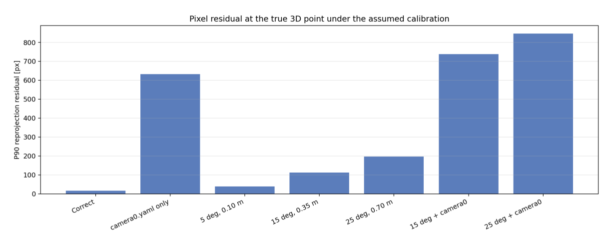


Figure 6. Hardware calibration ablation. Wrong intrinsics and marker-frame/optical-frame errors produce multi-meter PF errors and large reprojection residuals.

C. Practical Hardware Implications

The hardware result does not prove that calibration was the only issue in the real run; timing, detector quality, and marker visibility can still contribute. However, the ablation shows that calibration error is sufficient to explain the observed error scale. Before another hardware PF run, QTM marker poses should be treated as marker-body poses, not optical camera poses; a fixed $T_{\text{marker,optical}}$ hand-eye transform should be calibrated for each camera; and QTM drone positions should be reprojected into each camera as a pre-filter sanity check. If the projected point does not land near the detected drone, the filter should not be run yet.

VII. Proposed Body-Frame-Acceleration Lie-Group PF

The implemented PF is position/velocity based. To include more Lie-group structure in the prediction step, the next extension is to let each particle carry a latent inertial-navigation state. This follows the same

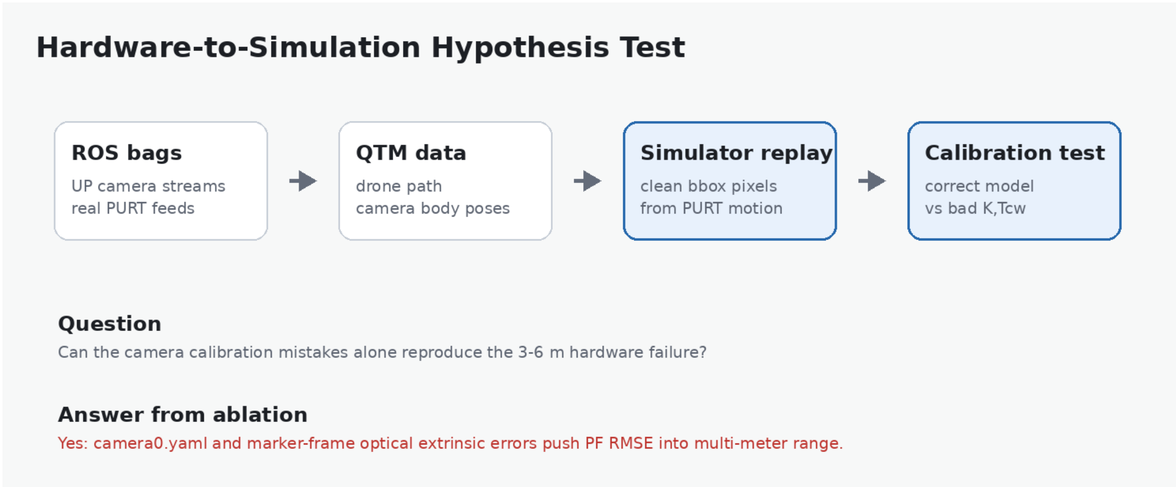


Figure 7. Hardware data in simulation: real ROS-bag snippets, QTM trajectory replay, simulated calibration ablation, and PF error metrics.

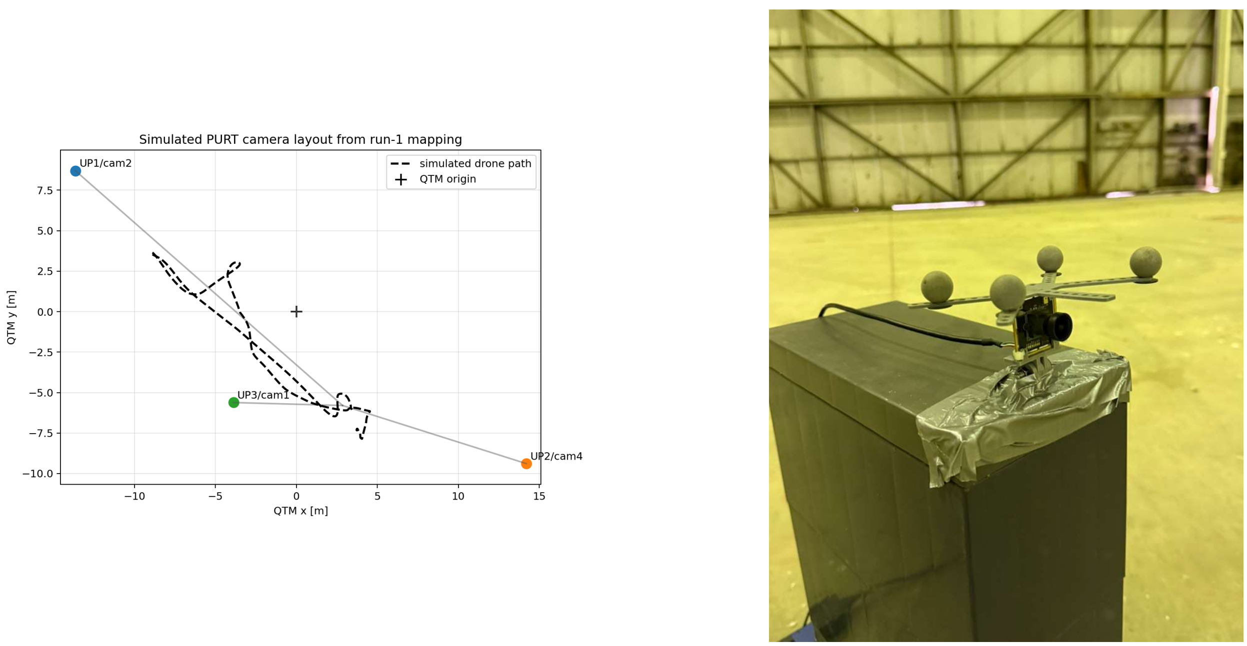


Figure 8. PURT-style camera layout and hardware camera with marker rig. The ablation models the effect of confusing the marker frame with the optical camera frame.

motivation as invariant filtering, where body-frame inputs and errors are naturally expressed on the group,[2] and connects to particle filtering on Lie groups:[4,5]

$$X_k^{(i)} = \left(R_{wb,k}^{(i)}, v_{w,k}^{(i)}, p_{w,k}^{(i)}\right) \in \mathrm{SE}_2(3).$$

The output of interest remains the trajectory $p_{w,k}$, but $R_{wb,k}$ is carried so that body-frame IMU acceleration can be mapped into world-frame motion.

Let the body-frame IMU input be

$$u_k = (\omega_{m,k}, a_{m,k}),$$

and sample body-frame process noise

$$\eta_{\omega,k}^{(i)} \sim \mathcal{N}(0, Q_\omega), \qquad \eta_{a,k}^{(i)} \sim \mathcal{N}(0, Q_a).$$

No IMU bias states are included in this proposed version. The attitude prediction is

$$R_{wb,k+1}^{(i)} = R_{wb,k}^{(i)} \operatorname{Exp}_{SO(3)}\left((\omega_{m,k} + \eta_{\omega,k}^{(i)})\Delta t\right),$$

where the product is group composition on SO(3). The accelerometer is mapped into world frame by

$$a_{w,k}^{(i)} = R_{wb,k}^{(i)}\left(a_{m,k} + \eta_{a,k}^{(i)}\right) + g_w,$$

with g_w omitted if $a_{m,k}$ is already gravity compensated. The velocity and position are then propagated as

$$v_{w,k+1}^{(i)} = v_{w,k}^{(i)} + \Delta t\, a_{w,k}^{(i)},$$

$$p_{w,k+1}^{(i)} = p_{w,k}^{(i)} + \Delta t\, v_{w,k}^{(i)} + \frac{1}{2}\Delta t^2 a_{w,k}^{(i)}.$$

The camera update remains unchanged:

$$\hat{z}_{j,k}^{(i)} = \pi(K_j, T_{j,cw}\bar{p}_{w,k}^{(i)}).$$

Thus the proposed PF uses Lie groups in both places: SO(3) propagation and body-to-world acceleration mapping in prediction, and SE(3) camera projection in measurement.

VIII. Conclusions

The study shows that Lie-group camera projection is the right mathematical language for multi-camera UAV tracking: it keeps camera extrinsics explicit and prevents frame-convention ambiguity. In the implemented position/velocity tracking problem, the EKF-style filters are fastest and most accurate in nominal unimodal cases. The PF is not the universal winner, but it is the natural estimator when the posterior is non-Gaussian or multimodal, as shown by the candidate-set experiment. The hardware ablation supports the hypothesis that incorrect intrinsics and marker-frame/optical-frame extrinsic errors can explain multi-meter hardware PF errors. The next technical step is a body-frame-acceleration Lie-group PF in which each particle carries latent attitude on SO(3) or $\mathrm{SE}_2(3)$, while the reported estimate remains the UAV trajectory.

A. Projection Jacobian

Let

$$p_c = R_{cw}p_w + t_{cw} = \begin{bmatrix} X & Y & Z \end{bmatrix}^\mathsf{T}.$$

For

$$u = f_x X/Z + c_x, \qquad v = f_y Y/Z + c_y,$$

the projection derivative with respect to the camera-frame point is

$$J_\pi = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{bmatrix}.$$

Since $\partial p_c/\partial p_w = R_{cw}$, the measurement Jacobian for the position/velocity state is

$$H_k = \begin{bmatrix} J_\pi R_{cw} & 0_{2\times 3} \end{bmatrix}.$$

B. Iterated EKF Algorithm

Given $\hat{x}_k^-$, P_k^-, and stacked measurement z_k, initialize

$$\hat{x}_k^{(0)} = \hat{x}_k^-.$$

At each iteration ℓ, compute

$$\hat{z}_k^{(\ell)} = h(\hat{x}_k^{(\ell)}), \qquad H_k^{(\ell)} = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_k^{(\ell)}}.$$

The residual corrected back to the predicted state is

$$\tilde{r}_k^{(\ell)} = z_k - h(\hat{x}_k^{(\ell)}) + H_k^{(\ell)}(\hat{x}_k^{(\ell)} - \hat{x}_k^-).$$

Then

$$S_k^{(\ell)} = H_k^{(\ell)} P_k^- H_k^{(\ell)\mathsf{T}} + R_k,$$
$$K_k^{(\ell)} = P_k^- H_k^{(\ell)\mathsf{T}} \left(S_k^{(\ell)}\right)^{-1},$$
$$\hat{x}_k^{(\ell+1)} = \hat{x}_k^- + K_k^{(\ell)} \tilde{r}_k^{(\ell)}.$$

The procedure stops after a fixed number of iterations or when the update change is below a tolerance.

C. Left-Invariant Error for the Proposed Lie-Group PF

The proposed $\mathrm{SE}_2(3)$ PF does not need a left-invariant error to perform the particle update. The PF propagates particles directly and updates weights by likelihood. Nevertheless, a left-invariant error is useful for diagnostics and for describing particle spread on the group.

Given a weighted group mean $\bar{X}_k$, define the particle error

$$\eta_k^{(i)} = \bar{X}_k^{-1} X_k^{(i)}.$$

The tangent-space error is

$$\epsilon_k^{(i)} = \mathrm{Log}\left(\bar{X}_k^{-1} X_k^{(i)}\right)^\vee.$$

For $\mathrm{SE}_2(3)$, this contains attitude, velocity, and position error coordinates:

$$\epsilon_k^{(i)} \approx \begin{bmatrix} \delta\theta_k^{(i)} \\ \delta v_{b,k}^{(i)} \\ \delta p_{b,k}^{(i)} \end{bmatrix}.$$

The b subscript means that errors are expressed in the body frame of the reference mean; the stored particle states are still $v_w^{(i)}$ and $p_w^{(i)}$.

The local weighted Lie-group mean minimizes

$$\bar{X}_k = \arg\min_Y \frac{1}{2} \sum_i w_k^{(i)} \left\| \mathrm{Log}(Y^{-1} X_k^{(i)})^\vee \right\|^2.$$

Let

$$\epsilon_i = \mathrm{Log}(\bar{X}_k^{-1} X_k^{(i)})^\vee.$$

Perturb the mean by

$$Y = \bar{X}_k \,\mathrm{Exp}(\delta^\wedge).$$

Then

$$Y^{-1} X_k^{(i)} = \mathrm{Exp}(-\delta^\wedge) \bar{X}_k^{-1} X_k^{(i)}.$$

Using the first-order Baker-Campbell-Hausdorff approximation near the mean,

$$\mathrm{Log}(Y^{-1} X_k^{(i)})^\vee \approx \epsilon_i - \delta.$$

Thus the local cost becomes

$$J(\delta) \approx \frac{1}{2} \sum_i w_k^{(i)} \|\epsilon_i - \delta\|^2.$$

At the optimum, the derivative at $\delta = 0$ must vanish:

$$\left. \frac{\partial J}{\partial \delta} \right|_{\delta=0} = -\sum_i w_k^{(i)} \epsilon_i = 0.$$

Therefore,

$$\sum_i w_k^{(i)} \operatorname{Log}(\bar{X}_k^{-1} X_k^{(i)})^\vee \approx 0.$$

This is the Lie-group analogue of the Euclidean fact that weighted errors around the mean sum to zero.

References

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, Cambridge, MA, 2005.

[2] A. Barrau and S. Bonnabel, "The Invariant Extended Kalman Filter as a Stable Observer," *IEEE Transactions on Automatic Control*, Vol. 62, No. 4, 2017, pp. 1797–1812.

[3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, Cambridge, UK, 2004.

[4] C. Zhang, A. Taghvaei, and P. G. Mehta, "Feedback Particle Filter on Matrix Lie Groups," arXiv:1701.02416, 2017.

[5] C. Chahbazian, *Particle Filtering on Lie Groups: Application to Navigation*, Ph.D. thesis, Universite Paris-Saclay, 2023.

[6] Z.-P. Yang, *Unmanned Aerial System Tracking in Urban Canyon Environments Using External Vision Sensors*, M.S. thesis, Purdue University, 2022.